Experiment 3: Linear Regression Model for Forecasting Time Series Data

---

1. Importing Necessary Libraries

Explanation:

This section imports the necessary libraries for data handling, numerical operations, plotting, and machine learning. These libraries are essential for reading the dataset, preprocessing data, building the model, and evaluating performance.

Corresponding Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, TimeSeriesSplit
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

---

2. Loading the Dataset

Explanation:
The dataset is loaded from a CSV file into a pandas DataFrame. The dataset used contains COVID-19 case data over time. This data will be used to forecast the number of confirmed cases.

Corresponding Code:

```
file_path = "/content/time-series-19-covid-combined (1).csv"
df = pd.read_csv(file_path)
```

3. Data Preprocessing

Explanation:

- Convert the **Date** column to datetime format for proper time-series analysis.
- Sort the dataset by date to maintain the chronological order.
- Group the data by date to get the **total number of confirmed, deaths, and recovered cases** globally per day.
- Perform a **log transformation** on the confirmed cases to make exponential growth trends easier to model.
- Generate **lag features**, where each day's data depends on data from the past 7 days (weekly lags).

Corresponding Code:

```
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values(by='Date')

df_grouped = df.groupby('Date')[['Confirmed', 'Deaths',
'Recovered']].sum().reset_index()
df_grouped['Log_Confirmed'] = np.log1p(df_grouped['Confirmed'])

num_lags = 7
for i in range(1, num_lags + 1):
    df_grouped[f'Confirmed_lag_{i}'] = df_grouped['Confirmed'].shift(i)
    df_grouped[f'Deaths_lag_{i}'] = df_grouped['Deaths'].shift(i)
    df_grouped[f'Recovered_lag_{i}'] = df_grouped['Recovered'].shift(i)

df_grouped.dropna(inplace=True)
```

---

4. Feature Selection and Target Variable

Explanation:

- The features consist of **lagged confirmed, deaths, and recovered cases** from the past 7 days.
- The target variable is the **log-transformed confirmed cases**, helping the model capture exponential trends.

Corresponding Code:

```
features = [f'Confirmed_lag_{i}' for i in range(1, num_lags + 1)] + \
           [f'Deaths_lag_{i}' for i in range(1, num_lags + 1)] + \
           [f'Recovered_lag_{i}' for i in range(1, num_lags + 1)]
target = 'Log_Confirmed'
```

5. Splitting the Data into Training and Testing Sets

Explanation:

- The data is split using **TimeSeriesSplit**, which ensures that training data always precedes testing data.
- This is important in time series data to avoid data leakage (future data influencing past predictions).

Corresponding Code:

```
X = df_grouped[features]
y = df_grouped[target]
tscv = TimeSeriesSplit(n_splits=5)
```

---

6. Training the Linear Regression Model

Explanation:

- A **Linear Regression** model is created.
- The model is trained and validated on each fold generated by TimeSeriesSplit.
- This approach mimics real-time forecasting, training only on past data and validating on future data.

Corresponding Code:

```
model = LinearRegression()

for train_index, test_index in tscv.split(X):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    model.fit(X_train, y_train)
```

7. Making Predictions

Explanation:

- After training on each fold, predictions are made for the test set.
- Predictions are made in the **log scale**.

Corresponding Code:

```
y_pred = model.predict(X_test)
```

---

8. Model Evaluation

Explanation

- **Mean Absolute Error (MAE)**: Measures average prediction error.
- **Mean Squared Error (MSE)**: Measures average squared error.
- **R^2 Score**: Measures how much variance is explained by the model.

Corresponding Code:

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R² Score: {r2:.4f}")
```

9. Visualization of Actual vs Predicted Values

Explanation:

- After training on the full dataset, the final predictions are **inverse-transformed** (exponentiated) back to the actual case counts.
- A plot is generated showing **actual vs predicted cases** over time.

Corresponding Code:

```
X_final_test = X.iloc[test_index]
y_final_test = y.iloc[test_index]
y_final_pred = model.predict(X_final_test)

y_final_pred_actual = np.expm1(y_final_pred)
y_final_test_actual = np.expm1(y_final_test)

df_results = pd.DataFrame({'Date': df_grouped['Date'].iloc[-len(y_final_test):],
                           'Actual Confirmed': y_final_test_actual,
                           'Predicted Confirmed': y_final_pred_actual})

print(df_results.head())

plt.figure(figsize=(10, 5))
plt.plot(df_results['Date'], df_results['Actual Confirmed'], label='Actual Cases',
color='blue')
plt.plot(df_results['Date'], df_results['Predicted Confirmed'], label='Predicted
Cases', color='red', linestyle='dashed')
plt.xlabel('Date')
plt.ylabel('Confirmed Cases')
plt.legend()
plt.title('COVID-19 Confirmed Cases Prediction using Linear Regression')
plt.xticks(rotation=45)
plt.show()
```

Result

Thus, the linear regression model for forecasting time series data (COVID-19 confirmed cases) has been successfully developed and evaluated.